JAVASCRIPT

Datos

Expresiones Regulares

- Expresiones regulares: formato
- símbolos
- banderas
- propiedades:
  - source,
  - flags,
  - lastindex.
- metodo test
- conjunto de caracteres
- Anclas
- Cuantificadores

Profesora:
Elizabeth Gisela Oviedo Luna

Expresiones Regulares

Las expresiones regulares son un sistema para buscar, capturar o reemplazar texto utilizando patrones. Estos patrones permiten realizar una búsqueda de texto de una forma relativamente sencilla y abstracta.

Estos patrones se representan mediante una cadena de texto, donde ciertos símbolos tienen un significado especial.

Formato:

/expresión_regular/flags

- /: son los delimitadores.
- expresión_regular: texto con símbolos especiales.
- Banderas o flags: opcional, caracteres que indican cómo funcionará la expresión regular.

Símbolos básicos en expresiones regulares:

Punto (.) es un comodín, significa cualquier carácter (letra, número, símbolo…), pero que ocupe sólo 1 carácter.

Ejemplo: let exprReg = /l.na/

El texto "l.na" es una expresión regular que usa el carácter especial punto (.) y las letras son caracteres literales. Por lo tanto, las siguientes palabras:

- "luna", "lona", "lana", "lmna" concuerdan con la expresión.
- "nala", "lola": no concuerdan porque el orden no es el correcto.
- "llena" no concuerda dado que el comodín solo vale por un carácter y no dos.
- "l na" concuerda dado que el espacio en blanco también es un carácter
- "lna" no concuerda dado que no hay un carácter en el lugar del punto.
- "Lona" no concuerda porque no está todo en minúscula.
- "LAna" si concuerda porque no hay restricción sobre el comodín.

Barra inclinada (\): Carácter de escape.

Podríamos añadir un \ antes del punto, y así decir que literalmente es un punto en vez de cualquier carácter.

Ejemplo: let exprReg = /l\.na/

- "l.na" concuerda dado que es literalmente el punto.
- "luna" no concuerda.

Flags o banderas

Serie de caracteres que indican el comportamiento de la expresión regular.

Opciones:

- **g - global:** Búsqueda global. Permite seguir buscando coincidencias en lugar de pararse al encontrar la primera.
- **i - ignoreCase:** No distingue entre mayúsculas y minúsculas.

- **m – Multiline**: Multilínea. Permite a ^ y $ tratar los finales de línea \r o \n.
- **u** - Unicode: Unicode. Interpreta secuencias Unicode.
- **y** - sticky: Busca sólo desde la posición indicada por lastIndex.
- **s** - dotAll: Establece si \n, \r, separación de párrafo o separación de línea deberían considerarse.
- **d** - hasIndices: Establece si al ejecutar un .exec() el resultado deberá tener propiedad indices.

Cada uno de estos flags se pueden comprobar si se encuentran activos desde Javascript con su booleano asociado, una propiedad de la expresión regular.

```
expresion = /reg/;   // No tiene ningún flag activado
expresion = /reg/i;  // Tiene el flag "i" activado
expresion = /reg/ig; // Tiene el flag "i" y "g" activado
```

Para empezar nos centraremos solo en la bandera i que no distingue entre minúscula y mayuscula, por lo tanto "A" y "a" concordarían como la misma letra.

Propiedades de una expresión regular

Ejemplo:

```
let expresion= /L..a/ig;
```

Propiedad source

Devuelve la expresión regular original definida sin los flags.

```
console.log(expresion.source); //   L..a
```

Propiedad flags

Devuelve los flags activados en la expresión regular.

```
console.log(expresion.flags);  // gi
```

Propiedad lastIndex

Devuelve la posición donde detectó una ocurrencia en la última búsqueda. Se utiliza para lanzar búsquedas sucesivas y comprobar coincidencias.

Método test(texto)

Devuelve un booleano que indica si detecta coincidencia con el patrón definido en el "texto".

Comprueba si la expresión regular concuerda con alguna parte del texto pasado por parámetro.

```
let expreg = /l.na/i;

console.log(expreg.test("lona"));    // true
console.log(expreg.test("Luna"));    // true - flag i
console.log(expreg.test("Milena"));  // true
console.log(expreg.test("él nadó")); // true
console.log(expreg.test("la nada"));   // false
```

Conjunto de caracteres

Englobado entre corchetes [] significa un rango de caracteres. Cualquiera de los caracteres del interior de los corchetes.

Ejemplo: Vocales en minúscula o mayúscula.

```
let expreg = /[aeiou]/i;
console.log(expreg.test("a"));  // true
console.log(expreg.test("E"));  // true (vocal, flag i)
console.log(expreg.test("t"));  // false
```

Englobado en [^] significa que no exista cualquiera de los caracteres del interior de los corchetes.

```
let expreg = /[^aeiou]/i;
console.log(expreg.test("a"));  // false
console.log(expreg.test("E"));  // false (vocal, flag i)
console.log(expreg.test("t"));  // true
```

A los lados de | significa una alternativa: lo que está a la izquierda o lo que está a la derecha.

```
let expreg = /l[o|u]na/i;
console.log(expreg.test("lona"));  // true
console.log(expreg.test("Luna"));  // true
console.log(expreg.test("lana"));  // false
console.log(expreg.test("l na"));  // false
console.log(expreg.test("lna"));   // false
```

Con los paréntesis () puedes agrupar una parte más específica para indicar alternativas.

Rangos de carácteres

En el interior de los corchetes [], si establecemos dos caracteres separados por guion establece un rango de caracteres sin tener que escribirlos a todos explícitamente. Ejemplo:

- [0-9] o \d: Un dígito del 0 al 9. Es decir, Es lo mismo escribir [0-9] que \d. Algunos programadores encuentran más explicativa la primera forma y otros más cómoda la segunda.
- [^0-9]o \D: No exista un dígito del 0 al 9.
- [A-Z] : Letra mayúscula de la “A” a la “Z”. Sin la ñ o letras acentuadas.
- [a-z] : Letra minúscula de la “a” a la “z”. Sin la ñ o letras acentuadas.
- [A-Za-z0-9] o \w: Carácter alfanumérico (letra mayúscula, minúscula o dígito).
- [^A-Za-z0-9] o\W: No exista carácter alfanumérico (letra mayúscula, minúscula o dígito).
- [\t\r\n\f] o \s: Carácter de espacio en blanco (espacio, TAB, CR, LF o FF).
- [^ \t\r\n\f] o \S: No exista carácter de espacio en blanco (espacio, TAB, CR, LF o FF).

Anclas

Permiten delimitar los patrones de búsqueda e indicar si empiezan o terminan por caracteres concretos, siendo mucho más específicos al realizar la búsqueda:

Carácter ^ Significa que empieza por.

```
let expreg = /^lu/i;
console.log(expreg.test("luz"));  // true
console.log(expreg.test("Luna")); // true
console.log(expreg.test("lu"));   // true
console.log(expreg.test("lana")); // false
console.log(expreg.test("llu"));  // false
```

Carácter $ Ancla. Significa que acaba en.

```
let expreg = /na$/i;
console.log(expreg.test("una"));   // true
console.log(expreg.test("NA"));    // true
console.log(expreg.test("nata"));  // false
console.log(expreg.test("na   "));  // false
console.log(expreg.test("Kaná"));    // false
```

Carácter \b: Límite de una palabra separada por espacios, puntuación o inicio/final.

```
let expreg = /na\b/i;
console.log(expreg.test("una"));   // true
console.log(expreg.test("semana corta"));  // true
console.log(expreg.test("Un manantial"));  // false
console.log(expreg.test("Hermana! vení")); // true
```

Carácter \B: Opuesta al anterior. Posición entre 2 caracteres. Cuando el texto no esté delimitado por una palabra, puntuación o string en sí.

```
let expreg = /na\B/i;
console.log(expreg.test("una"));   // false
console.log(expreg.test("semana corta"));  // false
console.log(expreg.test("Un manantial"));  // true
console.log(expreg.test("Hermana! vení")); // false
```

Cuantificadores

Indican cuántas veces se puede repetir el carácter inmediatamente anterior.

asterisco (*): El carácter anterior puede aparecer 0 o más veces.

```
let expreg = /a*/;
console.log(expreg.test("aaaa"));  // true
console.log(expreg.test("a"));  //  true
console.log(expreg.test("manantial")); // true
console.log(expreg.test("Este")); // true
```

suma (+): El carácter anterior puede aparecer 1 o más veces.

```
let expreg = /a+/;
console.log(expreg.test("aaaa"));  // true
console.log(expreg.test("a"));  //  true
console.log(expreg.test("manantial")); // true
console.log(expreg.test("Este")); // false
```

signo pregunta (?). El carácter anterior es opcional, puede aparecer o no.

```
let expreg = /nas?\b/;
console.log(expreg.test("runas"));  // true
console.log(expreg.test("una vida")); //  true
console.log(expreg.test("canasto")); // false
```

N veces {n}. El carácter anterior aparece n veces. Número exacto.

```
let expreg =  /[1-7]{2}/;
console.log(expreg.test("el 13")); //true
console.log(expreg.test("58 ")); // false
console.log(expreg.test("123 ")); // true
console.log(expreg.test("2")); // false
```

Si además lo delimitamos

```
let expreg =  /^[0-9]{2}$/;
console.log(expreg.test("el 13")); //false
console.log(expreg.test("58")); //true
console.log(expreg.test("123")); // false
console.log(expreg.test("2")); // false
```

N veces o más {n,}: El carácter anterior aparece n o más veces. Al menos n veces.

```
let expreg =  /^[0-9]{2,}$/;
console.log(expreg.test("13 ")); //false
console.log(expreg.test("58"));  //true
console.log(expreg.test("123")); //true
console.log(expreg.test("2"));  //false
```

N o M veces {n,m} El carácter anterior aparece de n a m veces.

```
let expreg =  /^[0-9]{2,4}$/;
console.log(expreg.test("1")); //false
console.log(expreg.test("58"));  //true
console.log(expreg.test("123")); //true
console.log(expreg.test("2452"));  //true
console.log(expreg.test("24523"));  //false
```

Ejemplo: Si queremos detectar todos los nombres que empiecen con las primeras 4 consonantes del abecedario y finalice con “a” u “o”.

```
let nombres = ['Blas', 'Ariel', 'Carla', 'Dante', 'Ana', 'Franco', 'Carlos'];
let lista= "Cumplen con la restricción:";

nombres.forEach(function (nom) {
  let regex = /^(b|c|d|f).+(a|o)$/i;
  if (regex.test(nom)) {
    lista += ` ${nom}`;
  }
});
console.log(lista);
```

```
Cumplen con la restricción:  Carla Franco
```

Se podria realizar de otra forma, pero la comprobación con expresiones regulares es mucho más simple cuando las condiciones son múltiples.